

Recursion basics

29 May 2026 16:16

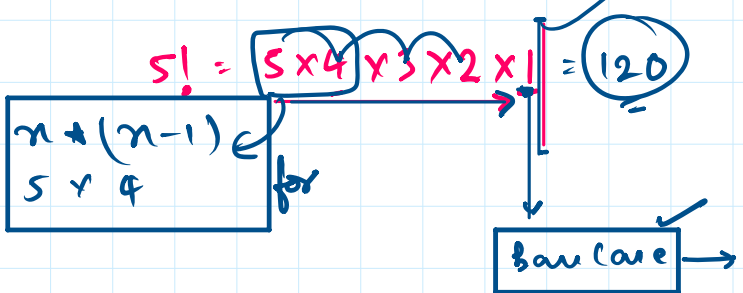
- 1) $f(x)$ which calls itself $\rightarrow$ again & again. until base case is satisfied
- 2) solving big problem $\rightarrow$ } recursion tree

topics:

- 1) Base case ✓
- 2) Recursive case ✓
- 3) Recursive tree ✓

Recursion uses $\rightarrow$ stack $\rightarrow$ LIFO, pop(), push(), top()...

1) factorial $\rightarrow$ 12th maths



factorial(5)

int result = 1;

for (int i = 1; i <= 5; i++) {

result = result * i;

1 * 1

1 * 2

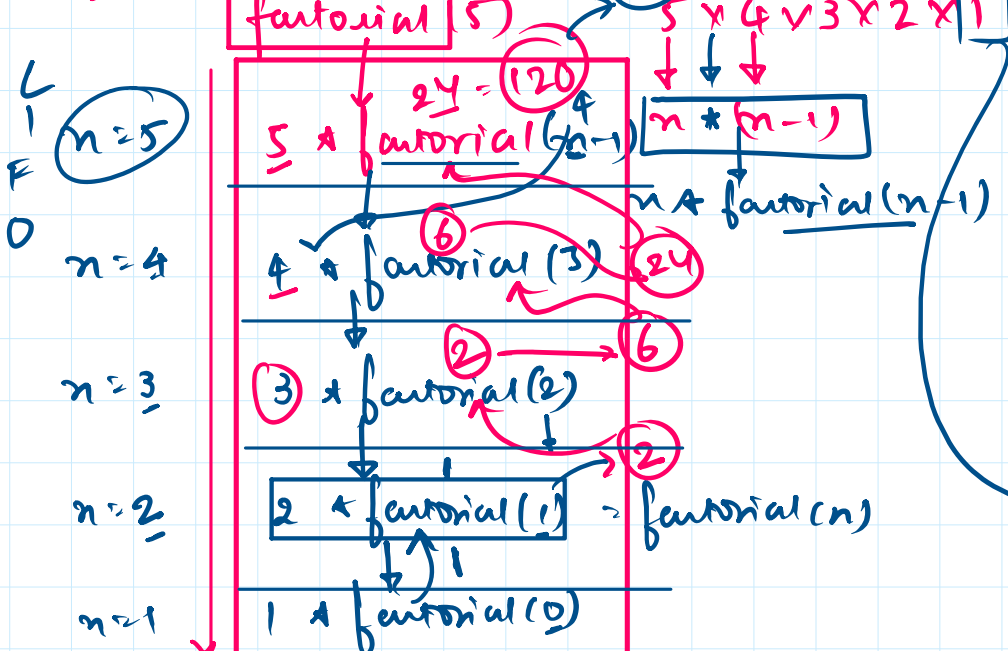
2 * 3

6 * 4

24 * 5

120

* Tree



if base case;

T	T	T
F	F	F
T	F	T
F	T	T
F	F	F

if (n == 0 || n == 1) {

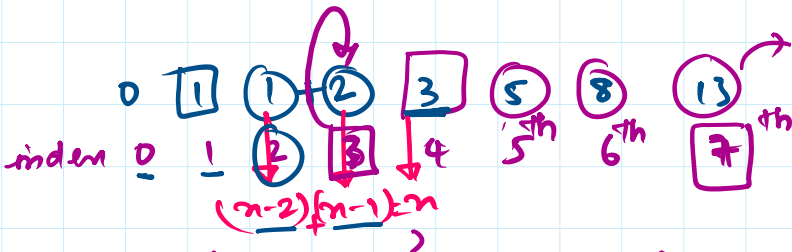
return 1;

}

$n=1$ $\downarrow$ $\boxed{1 + \text{factorial}(0)}$

$\overline{\quad}$ $\sim$

Eg: fibonacci



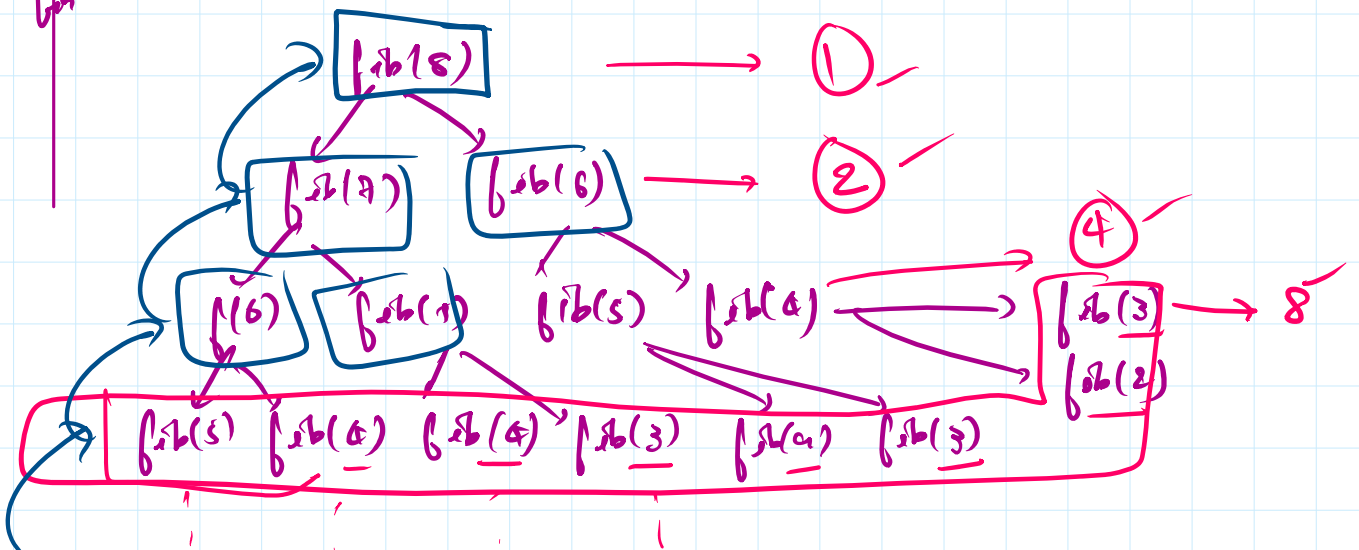
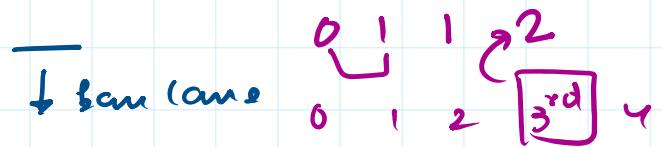
$\boxed{\text{fib}(n-2) + \text{fib}(n-1) = n}$

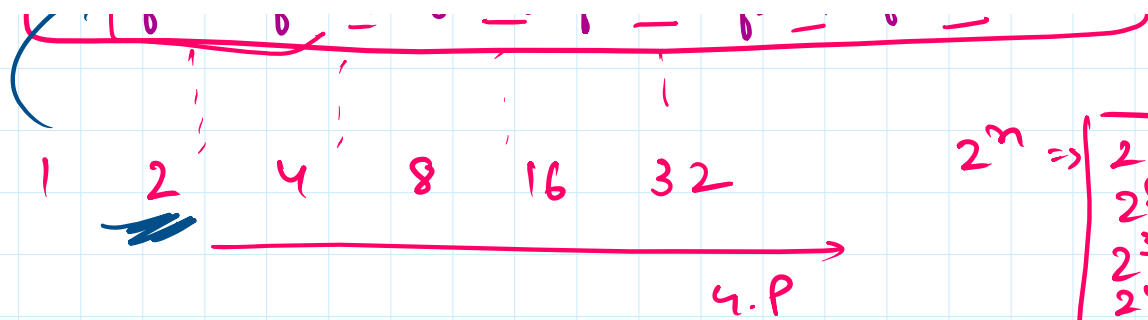
result

tree ✓
code

$\text{if } (n==1 || n==0) \{$
 $\text{return } n$

$T(n) = 2^n$





$$2^n \Rightarrow$$

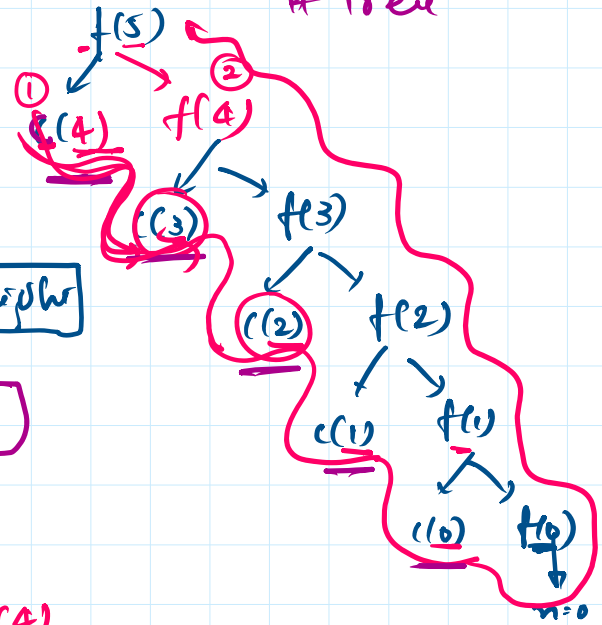
2^1	→	2
2^2	→	4
2^3	→	8
2^4	→	16
2^5	→	32

$\vdots$
 $\rightarrow 2^n$
 $ba \rightarrow \boxed{T.C(2^n)}$

```
main.cpp
1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4 void f(int n){
5     if(n > 0){
6         cout << n-1;
7         f(n-1);
8     }
9 }
```

GATE, 2m, 1m

Tree



if(n > 0)

top left bottom right

4 3 2 1 0

op:

① 5, 4, 3, 2, 1

② 4, 3, 2, 1, -

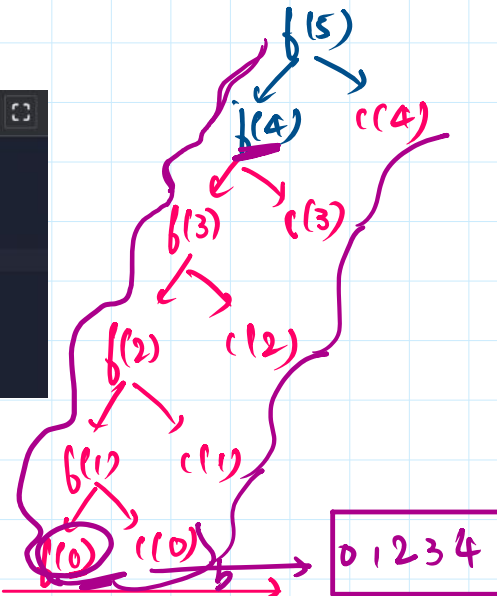
③ 4, 3, 2, 1, 0 ✓

```
main.cpp
1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4 void f(int n){
5     if(n > 0){
6         f(n-1);
7         cout << n-1;
8     }
9 }
```

op: 3 2 1

④ 0 1 2 3 4

n > 0
0 > 0



0 1 2 3 4

1 min

```
1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4 void f(int n){
5     if(n > 0){
6         cout << n;
7         f(n-1);
8         cout << n;
9     }
10 }
```

n=5

n-1)

PG CAT
GATE, Gout

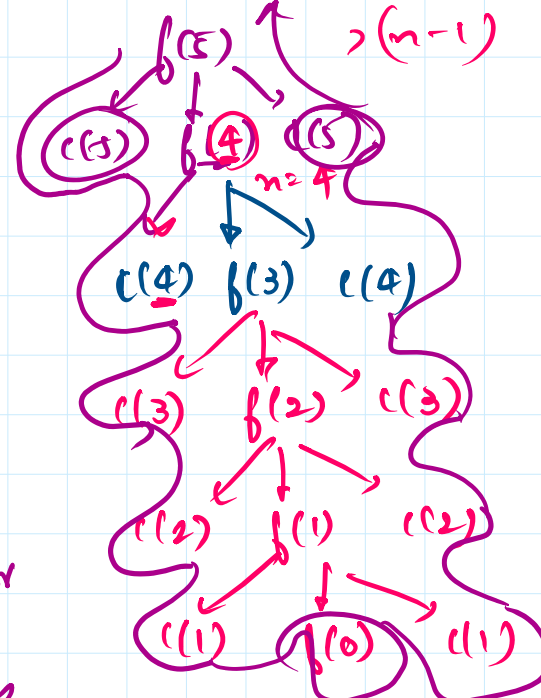
op: 0 1 2 3 4 5

5 4 3 2 1 1 2 3 4 5 ***

5 4 3 2 1 1 2 3 4 5

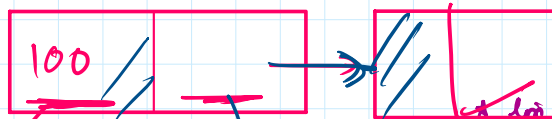
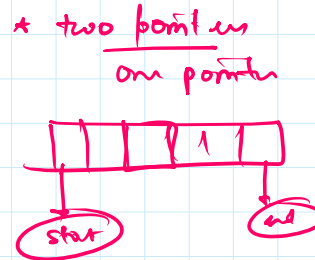
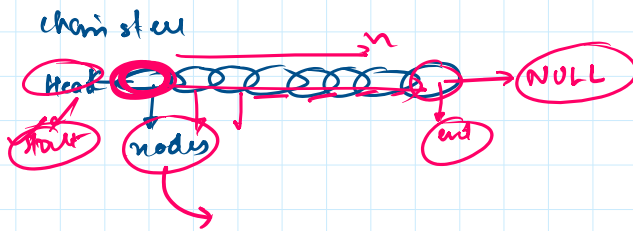
Print
Sum
Repeat

Tree



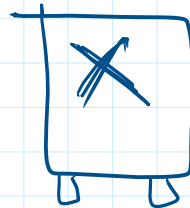
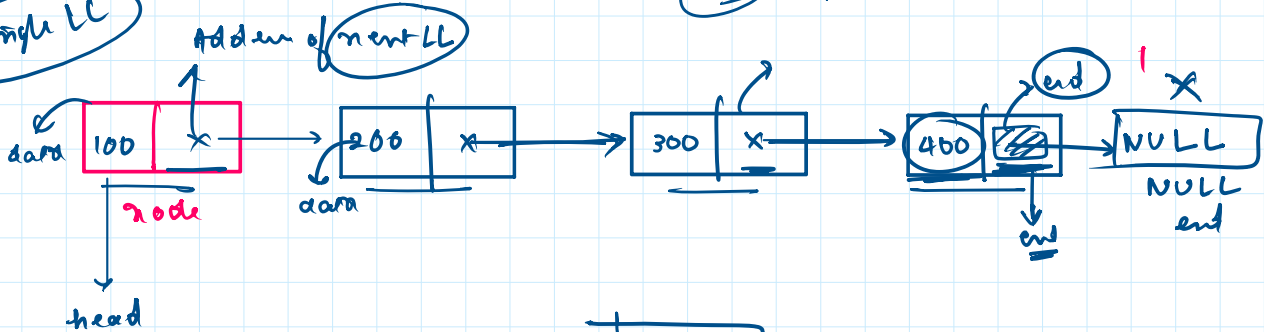
What is linked list??

- * linear ds
- * elements in Node
- * element pointer the next element
- * data, pointer to the next data.

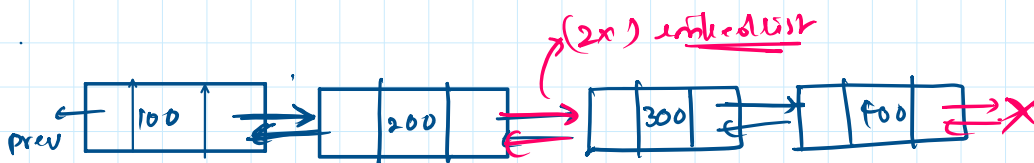


- * linear ds
- * elements in Node
- * element pointer the next element
- * data, pointer to the next data.

* Single LL

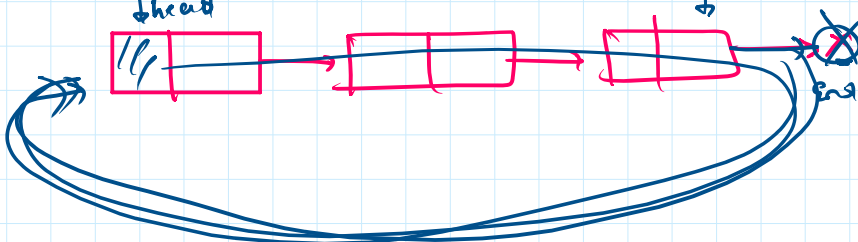


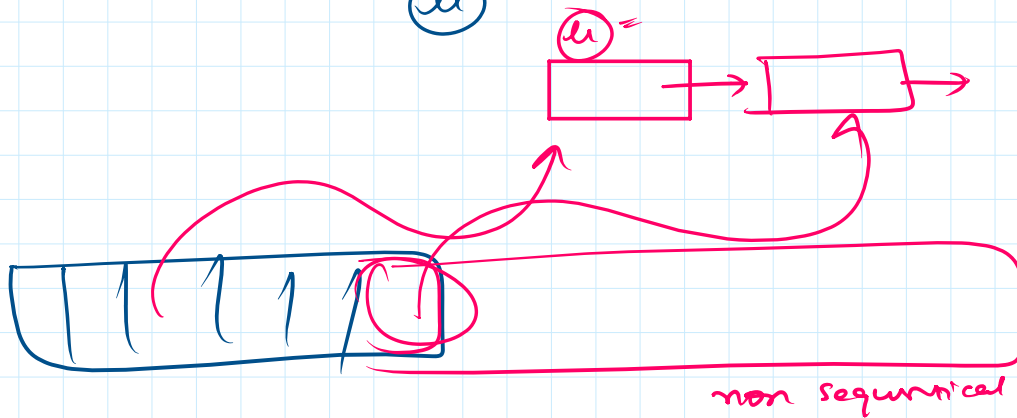
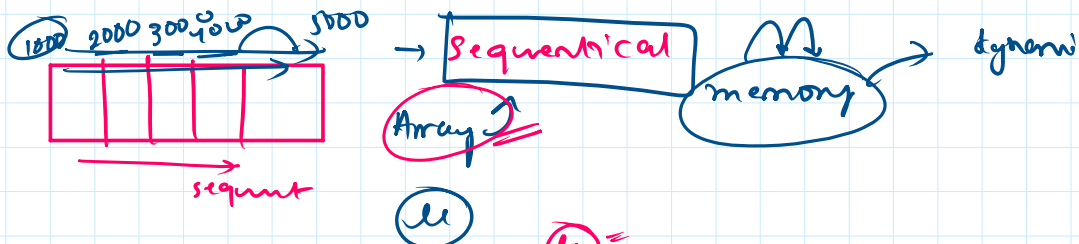
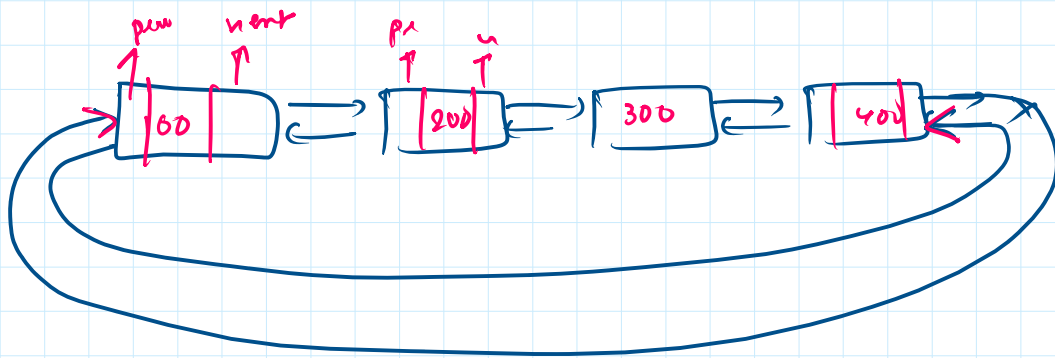
- Singly linked list
- Doubly linked list
- Circular linked list
- Doubly circular linked list



* singly + circular

last node will be connected to the head node

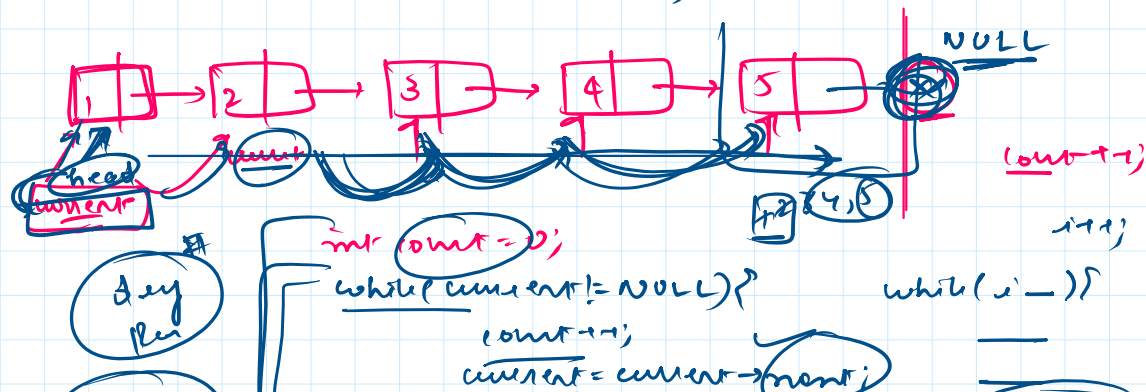
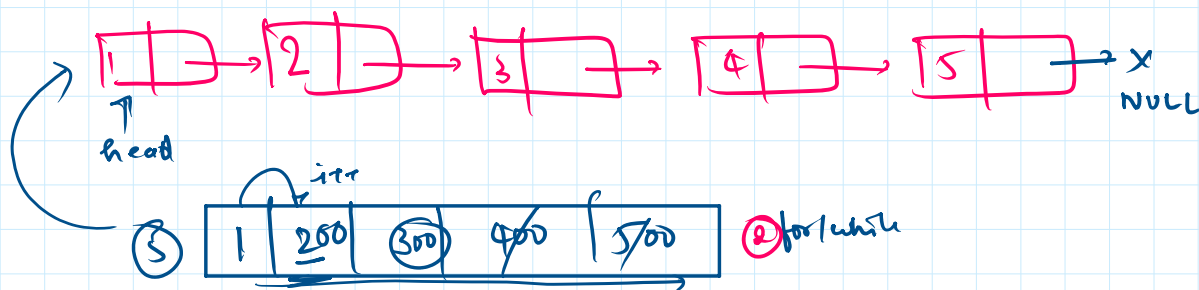




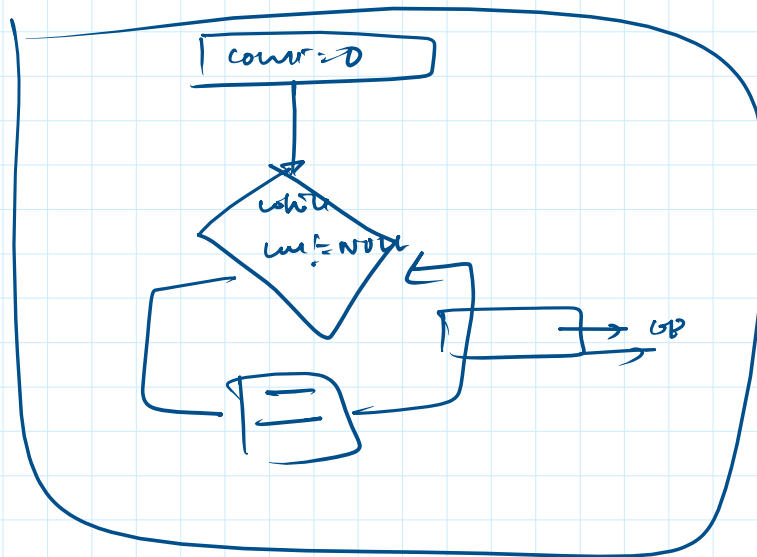
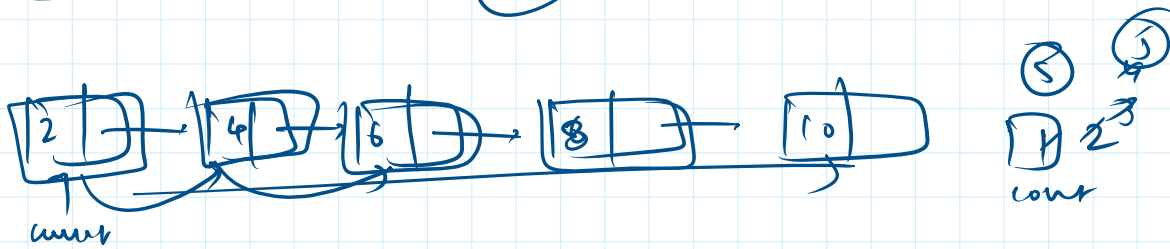
Input: head: 1 -> 2 -> 3 -> 4 -> 5
 head → 1 → 2 → 3 → 4 → 5 → NULL
 Output: 5
 Explanation: Length of the linked list is 5, as there are 5 nodes present in it.

Input: head: 2 -> 4 -> 6 -> 7 -> 5 -> 1 -> 0
 head → 2 → 4 → 6 → 7 → 5 → 1 → 0 → NULL
 Output: 7
 Explanation: Length of the linked list is 7, as there are 7 nodes present in it.

① 2 3 4 5
 count = 1
 length / count



prev
 prev code
 count++;
 current = current->next;
 prev = current;
 count++;



Input: head → 1 → 2 → 3 → 4 → 5 → NULL
 Output: 3
 Explanation: The given linked list is 1->2->3->4->5 and its middle is 3.
 head → 1 → 2 → 3 → 4 → 5 → NULL

0 1 2 3 4
 mid
 count/2 = 2

$$\frac{\text{start} + \text{end}}{2} = \text{mid}$$

time complexity

Bubble sort. $\rightarrow O(n^2)$

Insertion sort. $\rightarrow O(n^2)$

Selection sort. $\rightarrow O(n^2)$

Quick sort. $\rightarrow O(\log n)$

Merge sort. $\rightarrow O(n \log n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

$O(n)$

for $i=0$
for $j=0$
 $O(n^2)$

$O(n)$

$O(\log n)$

$swap(arr[j], arr[j+1])$

$10 > 1$
 $1 < 10$ x

$if(arr[j] > arr[j+1])$

$6 > 10$
 $10 > 9$

$10 > 14$
 $14 > 10$
 $10 > 6$
 $10 > 7$
 $10 > 1$
 j

10 1 7 6 14 9

1 2 3 4 5

i j $j+1$

1 10 7 6 14 9

i j $j+1$

1 7 10 6 14 9

i j $j+1$

1 7 6 10 14 9

i j $j+1$

1 7 6 10 14 9

i j $j+1$

1 7 6 10 14 9

i j $j+1$

1 7 6 10 14 9

i j $j+1$

1 7 6 10 14 9

i j $j+1$

1 6 7 9 10 14

i j $j+1$

10 1 7 6 14 9

i j $j+1$

10 7 6 14 9

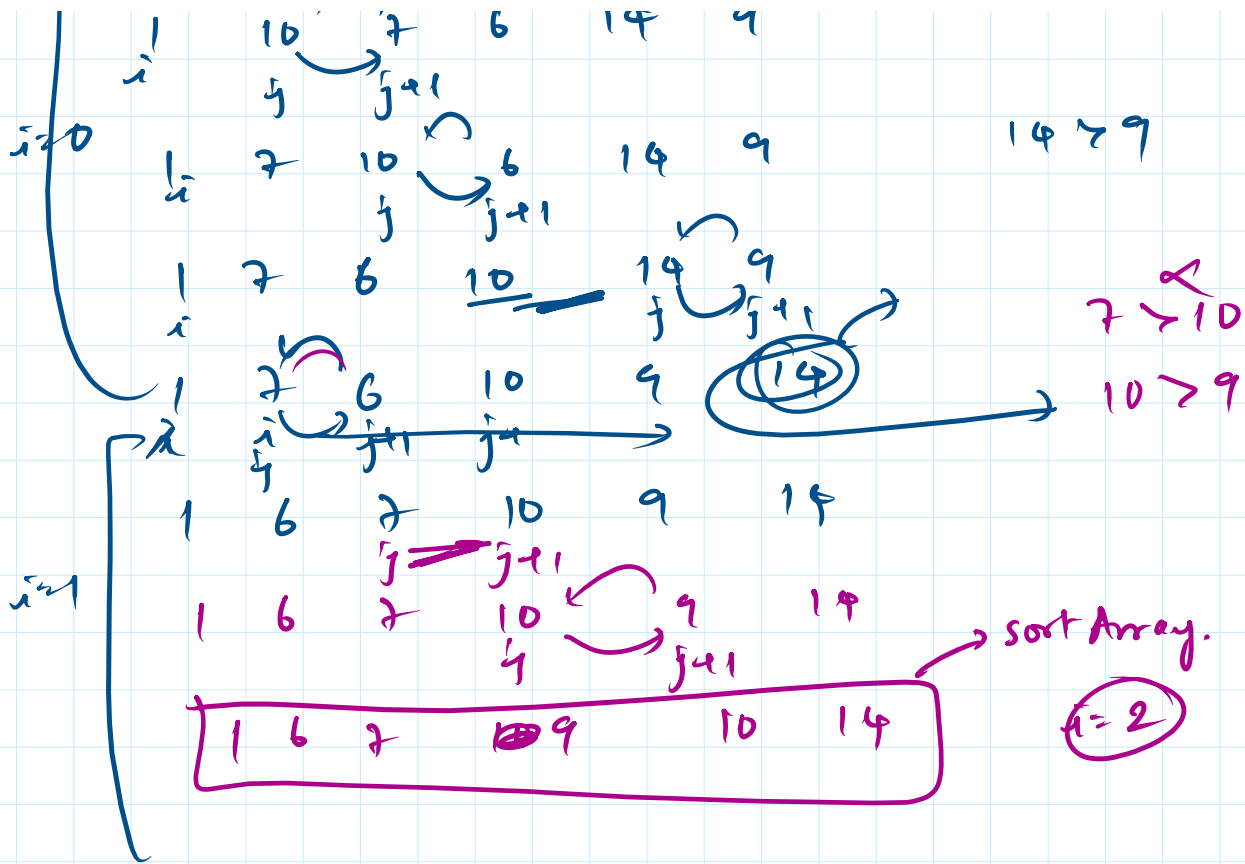
i j $j+1$

10 7 6 14 9

i j $j+1$

10 7 6 14 9

i j $j+1$



return nothing $\rightarrow$ if^*

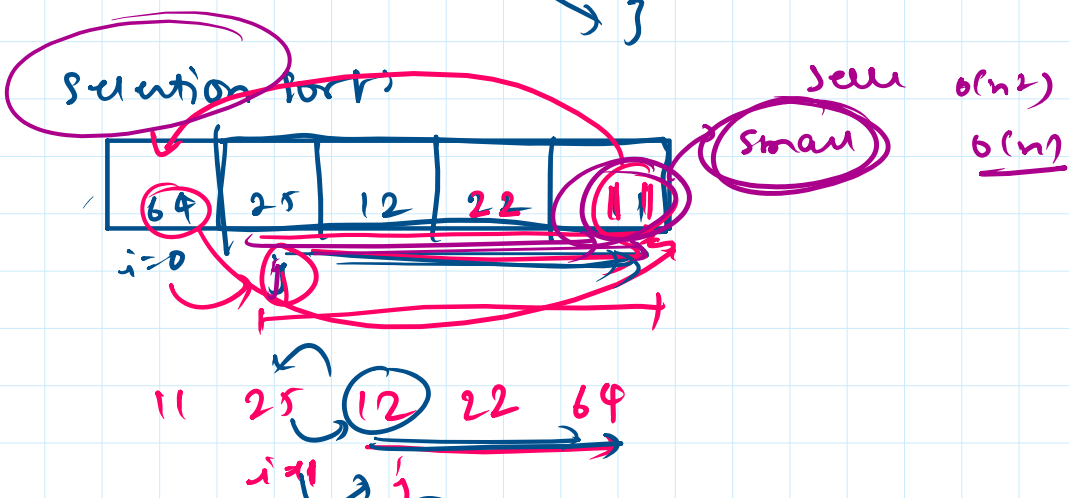
```

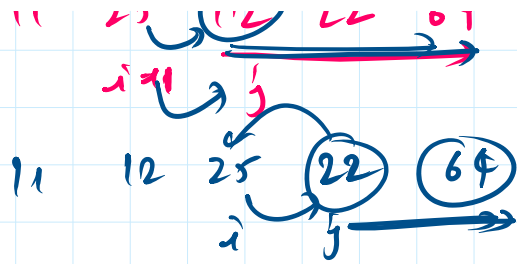
void bubbleSort(int arr[]) {
    int n = arr.size();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

```

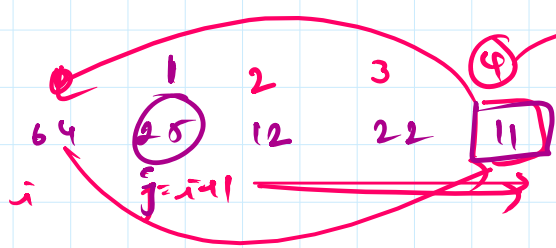
in sort, 2, $\rightarrow$ sort =

$O(n^2)$ $\leftarrow$ programming.

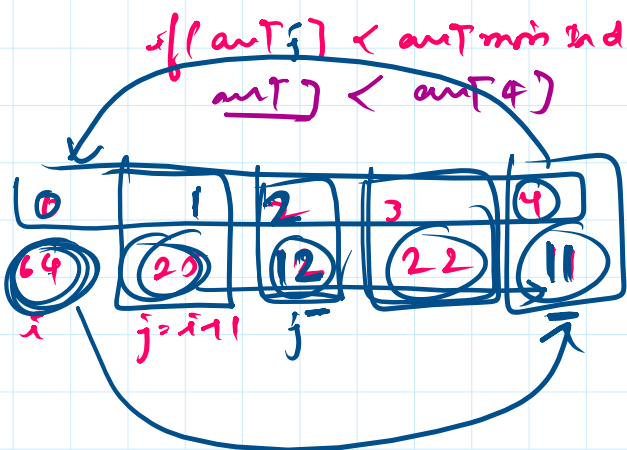




11 12 22 25 64 sorted Array



4
minIndex
swap(arr[minIndex], arr[i])
arr[4], arr[0]



4
minIndex
arr[minIndex]

Answers for the Quiz

06 June 2026

17:12

Question 4 of 4 · Score: 3

Selection Sort

What is the time complexity of selection sort in all cases (best, average, worst)?

- ☐ A $O(n)$
- ☐ B $O(n \log n)$
- ☒ C $O(n^2)$
- ☐ D $O(\log n)$

Explanation: Selection sort always performs exactly $n(n-1)/2$ comparisons regardless of input order, because it must scan the entire unsorted portion every pass to find the minimum. This gives $O(n^2)$ in all cases — it cannot be optimized like bubble sort.

[< Prev](#)

[See results ↗](#)

Selection Sort

What does selection sort do in each iteration?

- ☐ A Swaps adjacent elements that are out of order
- ☐ B Selects the maximum element and places it at the end
- ☒ C Finds the minimum element in the unsorted portion and places it at the beginning of that portion
- ☐ D Merges two sorted halves into one

Explanation: In each pass, selection sort scans the unsorted portion of the array, finds the minimum element, and swaps it with the first element of the unsorted portion — growing the sorted portion by one each time.

← Prev

Next →

Bubble Sort

What is the key operation performed in each pass of bubble sort?

- ☐ A Find the minimum element and place it at the front
- ☒ B Compare adjacent elements and swap them if they are in the wrong order
- ☐ C Divide the array into two halves and sort each half
- ☐ D Insert each element into its correct position

Explanation: Bubble sort repeatedly compares adjacent pairs of elements and swaps them if the left one is greater than the right. After each pass, the largest unsorted element 'bubbles up' to its correct position at the end.

← Prev

Next →

RECURSION BASICS

Which data structure is implicitly used during recursion?

☐ A Queue

☐ B Heap

☒ C Stack

☐ D Array

Explanation: Each recursive call is pushed onto the call stack. When the base case is reached, the calls are popped and resolved in LIFO order — just like a stack.

← Prev

Next →

RECURSION BASICS

What happens if a recursive function has no base case?

- ☐ A It returns 0 by default
- ☒ B It runs forever and causes a stack overflow
- ☐ C It automatically terminates after 100 calls
- ☐ D It skips the recursive call

Explanation: Without a base case, the function keeps calling itself, filling up the call stack until the program crashes with a stack overflow error.

← Prev

Next →

Question 1 of 10 · Score: 1

RECURSION BASICS

What is recursion in programming?

- ☐ A A loop that runs a fixed number of times
- ☒ B A function that calls itself to solve a smaller version of the same problem
- ☐ C A method of sorting arrays
- ☐ D A technique to allocate memory dynamically

Explanation: Recursion is when a function calls itself with a simpler or smaller input, breaking a problem down until it reaches a base case that can be solved directly.

← Prev

Next →

7. Which of the following is a defining characteristic of a circular singly linked list?

A. It requires twice as much memory as a standard singly linked list.

B. Every node points to every other node.

C. The 'next' pointer of the last node points back to the head node.

✓ Right answer (skipped)

This connection forms a closed loop, meaning there is no node pointing to null.

D. The list cannot be empty.

6. What distinguishes a doubly linked list from a singly linked list?

A. The last node points back to the first node.

B. It can store two different data types in each node.

C. It contains pointers to the head and tail nodes only.

D. Each node contains a pointer to the next node and a pointer to the previous node.

✓ **Right answer (skipped)**

Having a reference to both adjacent nodes allows traversal in both forward and backward directions.

1. Which of the following correctly describes the structure of a single node in a singly linked list?

A. It contains only data elements stored in contiguous memory.

B. It contains data and a pointer to the previous node.

C. It contains data and a pointer to the next node.

✓ **Right answer (skipped)**

The fundamental building block of a singly linked list holds the value and a reference to the subsequent element.

D. It contains a pointer to the head and a pointer to the tail.